OPERATING SYSTEMS

Midterm Exam

Answer Key

March 19th, 2025, from 09:00am to 10:00am

This is a **closed book exam**. All electronic devices are not permitted. **Do not** take pictures of the exam.

There are a total of **25 points**. You have **60 minutes** to write the exam.

The course instructor or the proctors **cannot answer questions** during the exam. If you find an exam problem unclear, then make reasonable assumptions, write down your assumptions, and solve the problem.

Do not attempt to cheat, copy, talk to a classmate, or give anything to or take anything from another classmate (borrowing a correction-pen maybe allowed), otherwise you may be **disqualified**.

Please write neatly. Write your answer using a pen or a pencil, but do not write in red or any color that is close to red. Use your rough sheet before answering.

Write your **full name** and **group** on the head of each of your answer sheets, and leave your student ID card out to be checked.

Full Name:		Grade:	Remarks:
Student #:	Group #:		

1. The following CISC x-86 assembly codes are to be run on an Intel 8086 microprocessor. The number of starts indicate the level of difficulty. Remember, `DIV CX` is equivalent to `AX = AX ÷ CX`. The `JMP LABEL` instruction allows a program to jump to an address of an instruction (to where the `LABEL` is defined) in the code. Also, the `JNO` jumps to the `LABEL` if there is no overflow detected. Answer the followoing questions [9.0 points]:

Code 1	Code 2	Code 3
★☆☆☆☆☆	★★☆☆☆☆	★★★☆☆☆

`[0xC11B] MOV AX, 0x000F`
`[0xC11C] MOV BX, 0x0014`
`[0xC11D] ADD AX, BX`
`[0xC11E] MOV CX, 0x0005`
`[0xC11F] DIV CX`
`[0xC120] INC AX`
`[0xC121] MOV [0x3E6F], AX`
`[0xC122] MOV AX, 0X0008`
`[0xC123] MOV CX, 0x0009`
`[0xC124] MOV DX, 0X0055`
`[0xC125] HLT`
	`[0xA122] MOV AX, 0X0002`
	`[0xA123] MOV CX, 0x0002`
	`[0xA124] MOV DX, 0X0040`
	`[0xA124] LABEL:`
	`[0xA126]  DEC CX`
	`[0xA127]  MUL DX`
	`[0xA128]  JNO LABEL`
	`[0xA129] DIV CX`
	`[0xA12A] INC CX`
	`[0xA12B] DIV CX`
	`[0xA12C] JMP LABEL`
	`[0xA12D] HLT`
		`[0xE122] MOV AX, 0X0000`
		`[0xE123] MOV BX, 0x0001`
		`[0xE124] LABEL:`
		`[0xE125]  ADD AX, BX`
		`[0xE126]  INC BX`
		`[0xE127]  JNO LABEL`
		`[0xE128] HLT`

a) In Code 1, what is the value (in decimal) that will be stored in the memory location `0x3F6F` when the program counter (`PC`) points the value `0xC122`?

The value is `0x0008` *or* `8`

b) In Code 2, what is the value (in hexadecimal) that will be stored in register `CX` when the program counter (`PC`) points the value `0xA129`?

The value is `0xFFFF` *or* `65535`

c) In Code 3, what is the value (in decimal) that will be stored in register `BX` when the program counter (`PC`) points the value `0xE128`?

The value is `0x016A` *or* `362` (Here 2 points)

d) Which code among the above three code, terminates following an exception?

(here 0.5 per correct answer)

Code *2* *ran into an exception, and the exception was* *Division by Zero*

e) Assuming one instruction (i.e., Fetch, Decode, Execute — all together) take one clock cycle, how many clock cycles do the codes consume to terminate?

Code 1 takes *11* *cyles*

Code 2 takes *15* *cyles*

Code 3 takes *≈ 1089* *cyles* (Here 2 points)

2. Consider the following codes, where `fork()` is a system call to create a process. The number of starts indicate the level of difficulty. Assuming that all system calls (`fork()`, `execl()`, `printf()`, `return`) succeed, answer the following questions [6.0 points]:

Code 1	Code 2
★★★★★★	★★★★★★

`char main(int argc, char *argv[])`
`{`
`  for (char k='a'; k<'g'; k++)`
`  {`
`  fork();`
`  printf("Pumped...\n");`
`  printf("Letter %s \n", &k);`
`  k++;`
`  }`
`  return '0';`
`}`
	`void main(int argc, char *argv[]) {`
	`  if(fork())`
	`  {`
	`  printf("Pumped\n");`
	`  execl("/bin/ls", "ls", NULL);`
	`  printf("Pumped\n");`
	`  }`
	`  else {`
	`  if(!fork()) printf("Pumped\n");`
	`  }`
	`  return;`
	`}`

a) In Codes 1 and 2, how many processes will be created including the parent?

There will be 8 processes in Code 1, and 3 processes in Code 2

b) In Codes 1 and 2, how many times the message "`Pumped` ..." will be printed?

It will be printed 14 times in Code 1, and 2 times in Code 2

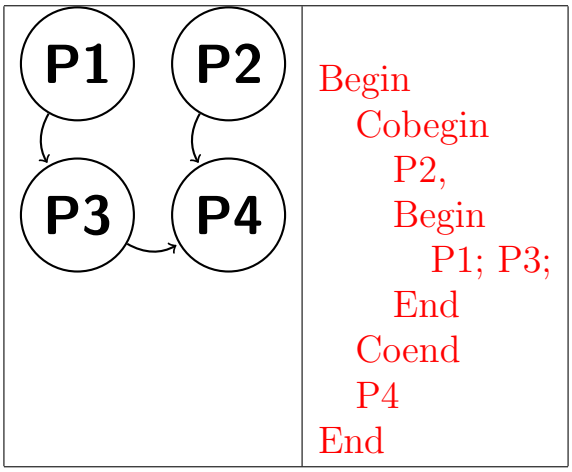
c) In Code 1, how many times the message "`Letter c`" will be printed?

The message "Letter c" will be printed 4 times

d) In Code 2, how many processes will not print anything?

Among all the processes, 1 will not print a message

3. Use Begin/End and CoBegin/CoEnd programming constructs to write code matching this precedence graph (use the empty space on the right of the PPG) [2.0 points].



4. Use **two** semaphores X and Y, to write code matching the precedence graph in Question 3. Note that for each semaphore that you use, it needs to be declared and initialized. Also, you are not allowed to merge codes or use busy-waiting. In addition to that, when using the primitives `P()` and `V()`, you are only allowed to call the primitive `P()` once in a given code, for a given semaphore variable [6.0 points].

   (1 point per answer)

Semaphore X=0;		Semaphore Y=-1;	
P1	P2	P3	P4
Code 1 V(X)	Code 2 V(Y)	P(X) Code 3 V(Y)	P(Y) Code 4 . . . . .
. . . . .	. . . . .	. . . . .	. . . . .

5. Consider the Peterson's algorithm for the critical section problem.

Process 0:

```
Begin
  while(true)
  {
     (1) flag[0]=true;
     (2) turn=1;
     (3) while(flag[1] && turn==1);
     (4) ...  Critical Section ...;
     (5) flag[0]=false;
  }
End
```

Process 1:

```
Begin
  while(true)
  {
     (1) flag[1]=true;
     (2) turn=0;
     (3) while(flag[0] && turn==0);
     (4) ...  Critical Section ...;
     (5) flag[1]=false;
  }
End
```

If the two statements (1) and (2) are swapped in both processes (i.e., you swap within a given code), state whether the altered code still satisfies the three requirements of the critical section problem. Explain which requirement has been violated [2.0 points].

The mutual exclusion is violated: P0 makes turn=1, then get context-switched by P1. P1 makes turn=0 and flag[1]=true, then enters the critical section. P1 gets context-swtiched by P0. P0 makes flag[0]=true, and enters the critical section too, as turn was set to 0 by P1.